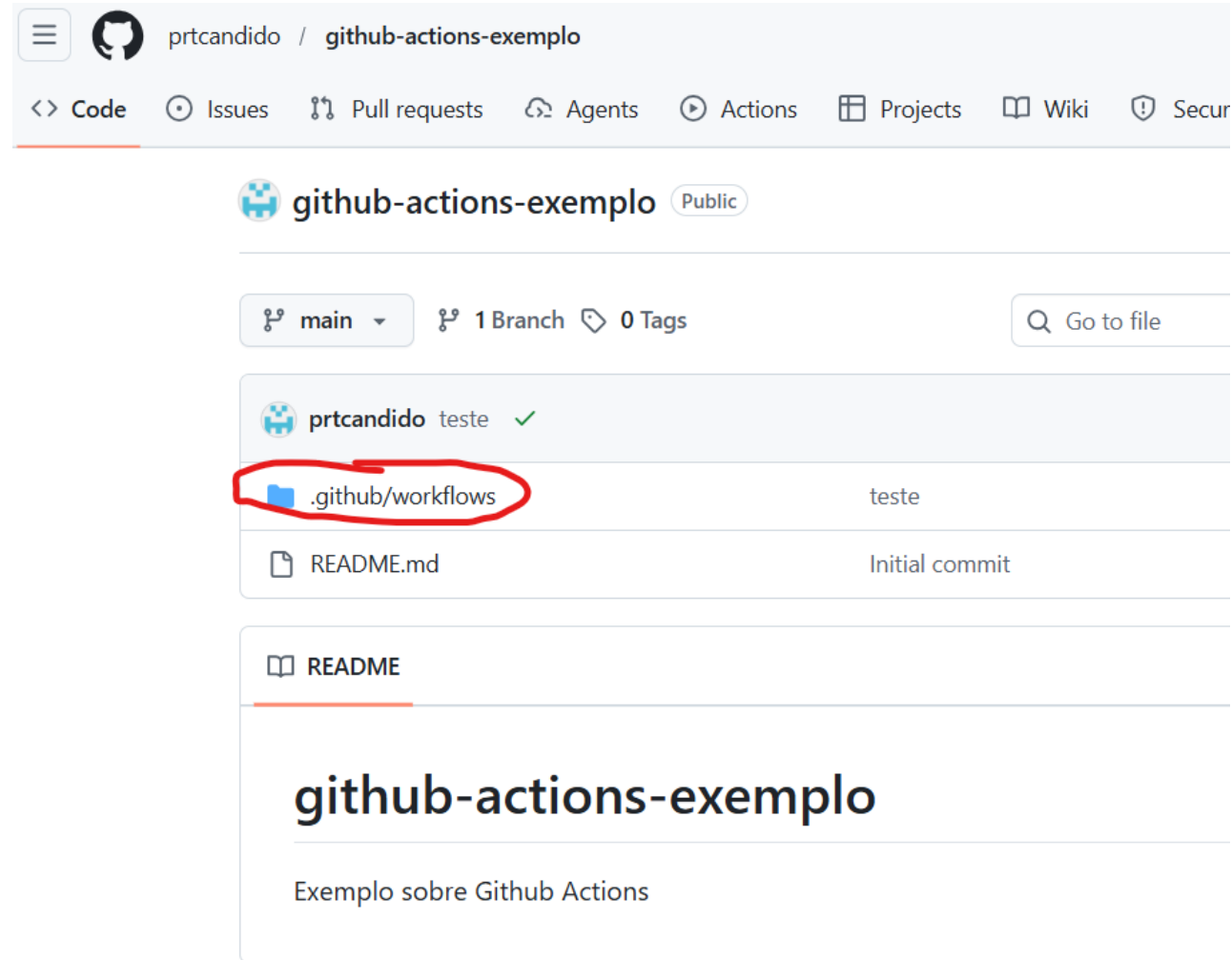


CI/CD

Github Action
Introdução

PAULO R T CÂNDIDO

Criar pasta workflows



prtccandido / github-actions-exemplo

<> Code Issues Pull requests Agents Actions Projects Wiki Secur

github-actions-exemplo Public

main 1 Branch 0 Tags Go to file

prtccandido teste ✓

.github/workflows teste

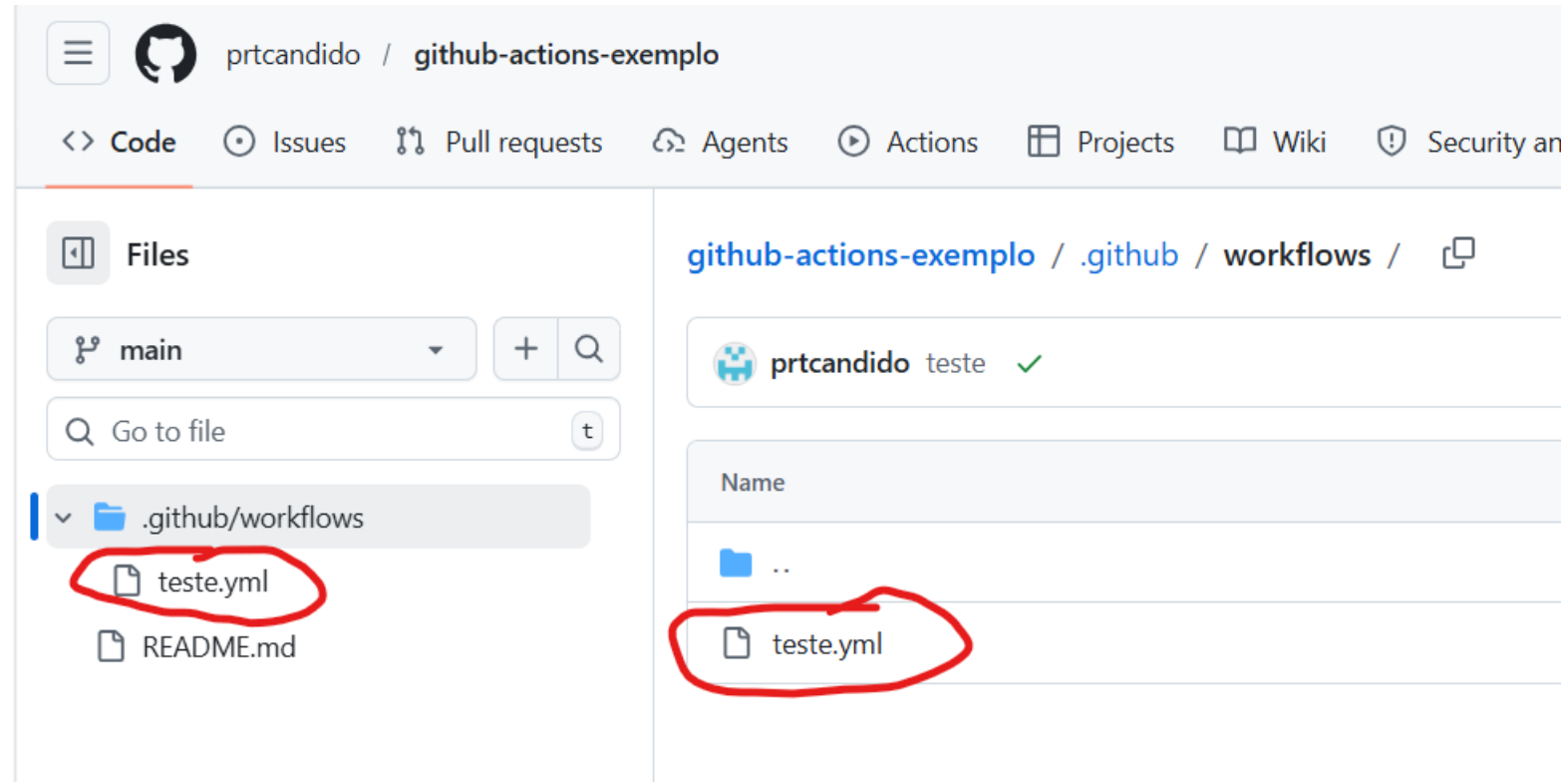
README.md Initial commit

README

github-actions-exemplo

Exemplo sobre Github Actions

Arquivo do Workflow (pipeline)



Código do Workflow

[github-actions-exemplo/.github/workflows/teste.yml at main · prtcandido/github-actions-exemplo](https://github.com/prtcandido/github-actions-exemplo/blob/main/.github/workflows/teste.yml)

Identação com dois espaços em branco – não usar TAB

```
name: Meu Workflow      # Nível 0
on: [push]             # Nível 0

jobs:                  # Nível 0
  build:                 # Nível 1 (2 espaços)
    runs-on: ubuntu-latest # Nível 2 (4 espaços)
    steps:               # Nível 2 (4 espaços)
      - name: Checkout    # Nível 3 (6 espaços - o hífen conta!)
        uses: actions/checkout@v4
```

Extensão VSC: YAML da Red Hat

Exemplo de Eventos

push: O mais comum. Dispara quando alguém envia código. Você pode filtrar por branches ou caminhos de arquivos.

YAML

on:

push:

branches: [main, develop]

paths:

- 'src/**' # Só dispara se mudar algo na pasta src

pull_request: Ideal para rodar testes ou builds de verificação antes de aceitar um código na branch principal.

YAML

on:

pull_request:

types: [opened, synchronize, reopened]

Exemplo de Eventos

Eventos Agendados (Scheduled Events)

Permitem que o workflow rode em horários específicos, como um "Cron job". Útil para backups, limpezas de logs ou builds noturnos.

schedule: Usa a sintaxe POSIX cron.

YAML

on:

schedule:

- cron: '0 0 * * *' # Roda todo dia à meia-noite (UTC)

Horários fixos comuns

Toda segunda-feira às 08:00 (UTC):

cron: '0 8 * * 1' (O número 1 representa segunda-feira).

Todo dia 1º do mês ao meio-dia:

cron: '0 12 1 * *'

A cada 15 minutos:

cron: '*/15 * * * *'

Exemplo de Eventos

Eventos Manuais e Externos

Às vezes você quer ter o controle total de quando a automação deve rodar, ou quer que um sistema externo avise o GitHub.

`workflow_dispatch`: Adiciona um botão "Run workflow" na interface do GitHub. Você pode até definir campos de entrada (inputs).

Exemplo no próximo slide.

`repository_dispatch`: Disparado via API do GitHub. Útil quando você quer que uma ferramenta externa (um sistema de monitoramento ou outro script) inicie o processo.

Exemplo de Eventos

on:

workflow_dispatch:

inputs:

ambiente:

description: 'Onde fazer o deploy?'

required: true

default: 'homologacao'

jobs:

deploy-job:

runs-on: ubuntu-latest

steps:

- name: Verificando o input

run: |

echo "Iniciando o deploy para o ambiente: \${github.event.inputs.ambiente}"

- name: Exemplo de condicional baseado no input

if: \${github.event.inputs.ambiente == 'producao' }

run: echo "CUIDADO: Rodando scripts de produção!"

Exemplo de Eventos

Eventos de Sistema (Webhooks)

O GitHub pode reagir a quase tudo que acontece na plataforma:

Evento	Quando ocorre
release	Quando você publica uma nova versão oficial do seu software.
issues	Quando uma tarefa é aberta ou fechada.
watch	Quando alguém dá uma "estrela" no seu repositório.
workflow_run	Quando um workflow depende da conclusão de outro workflow.

Variáveis internas

name: Exemplo de Variaveis

1. Escopo Global (Disponível em todo o Workflow)

env:

GLOBAL_VAR: "Minha variavel global"

jobs:

meu-job:

runs-on: ubuntu-latest

2. Escopo do Job (Disponível apenas dentro deste Job)

env:

JOB_VAR: "Variavel do Job"

Variáveis internas

steps:

- name: Usando variaveis via Shell ([Linux/Bash](#))

3. Escopo do Step

env:

STEP_VAR: "Variavel do Step"

run: |

echo "Global: \$GLOBAL_VAR"

echo "Job: \$JOB_VAR"

echo "Step: \$STEP_VAR"

- name: Usando variaveis via sintaxe de expressão do GitHub

run: |

echo "Acessando via contextos: \${env.GLOBAL_VAR}"

Variáveis (repositório)

São usadas para armazenar dados que **não são secretos**, mas que você quer centralizar para facilitar a manutenção.

- **Visibilidade:** O valor é visível em texto claro nos logs do GitHub.
- **Exemplos:** Nome de um banco de dados, porta de conexão (3000), URL de uma API pública, ou o nome da imagem Docker.
- **Vantagem:** Se você mudar o nome do banco, altera em um só lugar (nas configurações do repositório) e todos os seus workflows se atualizam.

Acesso via YAML(Workflow)

```
vars.NOME_VAR
```

Secrets (repositório)

São usados para informações sensíveis que **nunca** devem ser expostas.

- Criptografia:** O GitHub criptografa esses valores. Assim que você salva, ninguém (nem você) consegue ver o valor novamente na interface.
- Mascaramento:** Se o seu script tentar dar um `echo` em um Secret, o GitHub exibirá `***` nos logs para proteger a informação.
- Exemplos:** Senhas de banco de dados, Tokens de API, Chaves SSH e o `DOCKERHUB_TOKEN`.

Acesso via YAML (Workflow)

`secrets.NOME_SEC`

Definindo Variáveis e Secrets

Pressione Settings no repositório e depois prossiga conforme imagem.

The screenshot displays the GitHub repository settings interface. On the left, a sidebar lists various settings categories. The 'Secrets and variables' category is selected and expanded, with the 'Actions' sub-option highlighted by a red circle. The main content area is divided into two tabs: 'Secrets' and 'Variables'. The 'Secrets' tab is active, showing 'Environment secrets' and 'Repository secrets' sections. Both sections indicate that there are no secrets currently defined. A 'Manage environment secrets' button is visible under the 'Environment secrets' section, and a 'New repository secret' button is visible under the 'Repository secrets' section. A 'Preview' button is also present in the sidebar next to the 'Models' option.

Access

- Collaborators
- Moderation options

Code and automation

- Branches
- Tags
- Rules
- Actions
- Models Preview
- Webhooks
- Copilot
- Environments
- Codespaces
- Pages

Security and quality

- Advanced Security
- Deploy keys
- * Secrets and variables**
 - Actions**
 - Codespaces
 - Dependabot

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Secrets **Variables**

Environment secrets

This environment has no secrets.

[Manage environment secrets](#)

Repository secrets

This repository has no secrets.

[New repository secret](#)

Exemplo uso Variáveis e Secrets

```
# Exemplo 2: Usando Secret em um comando de Shell direto
- name: Login em serviço externo
  run: |
    echo "Fazendo login..."
    login-tool --user ${ secrets.DOCKER_USER } --password ${ secrets.DOCKER_TOKEN }
```

Múltiplos Jobs

Jobs por padrão são executados em paralelo. No exemplo abaixo, o deploy executará ao mesmo tempo que o job testes (incorreto).

```
name: Jobs em Paralelo
```

```
on: [push]
```

```
jobs:
```

```
  job_testes:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - run: echo "Executando testes unitários..."
```

```
  job_deploy:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - run: echo "Fazendo deploy imediato!"
```

Múltiplos Jobs

Pode ser configurado que os Jobs sejam executados sequencialmente, um após o outro. Deve ser definido, usando "NEEDS", que o Job de deploy precisa que o de testes finalize com sucesso para que ele possa ser executado.

```
name: Jobs em Sequência
```

```
on: [push]
```

```
jobs:
```

```
  testes:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - run: echo "Rodando testes..."
```

```
      # Se o teste falhar o job de deploy não executa
```

```
      - run: exit 0
```

```
  deploy:
```

```
    runs-on: ubuntu-latest
```

```
    needs: testes # <--- O segredo está aqui!
```

```
    steps:
```

```
      - run: echo "Testes passaram! Iniciando deploy seguro..."
```

if/else (recomendado)

name: Deploy Pipeline

on:

push:

branches: ["main", "develop", "feature/*"]

jobs:

deploy:

runs-on: ubuntu-latest

steps:

- name: Checkout do Código

- uses: actions/checkout@v4

- # --- IF (Cenário 1: Produção) ---

- name: Deploy em Produção

- if: github.ref_name == 'main'

- run: echo "Iniciando deploy no ambiente de PRODUÇÃO"

- # --- ELSE IF (Cenário 2: Homologação) ---

- name: Deploy em Staging

- if: github.ref_name == 'develop'

- run: echo "Iniciando deploy no ambiente de STAGING"

- # --- ELSE (Cenário 3: Qualquer outra branch) ---

- name: Notificar Feature Branch

- if: github.ref_name != 'main' && github.ref_name != 'develop'

- run: echo "Validação concluída para a branch temporária: \${github.ref_name}"

if/else - Lógica em Shell com GITHUB_OUTPUT

jobs:

build:

runs-on: ubuntu-latest

steps:

- name: Determinar Configurações do Ambiente (If/Else via Shell)

id: set-env

run: |

if ["\${{ github.ref_name }}" = "main"]; then

echo "environment=production" >> \$GITHUB_OUTPUT

echo "retries=5" >> \$GITHUB_OUTPUT

elif ["\${{ github.ref_name }}" = "develop"]; then

echo "environment=staging" >> \$GITHUB_OUTPUT

echo "retries=3" >> \$GITHUB_OUTPUT

else

echo "environment=development" >> \$GITHUB_OUTPUT

echo "retries=1" >> \$GITHUB_OUTPUT

fi

- name: Executar Deploy no Ambiente Calculado

run: |

echo "Publicando no ambiente: \${{ steps.set-env.outputs.environment }}"

echo "Tentativas de reenvio: \${{ steps.set-env.outputs.retries }}"

if/else – nível de Job

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm test

  # Este Job só é executado IF os testes passarem E a
  # branch for a main
  deploy-production:
    needs: test
    if: github.ref_name == 'main'
    runs-on: ubuntu-latest
    steps:
      - run: echo "Executando deploy em produção..."
```

Funções úteis em condicionais

No GitHub Actions, as funções de status `success()`, `failure()`, `always()` e `cancelled()` são usadas dentro do bloco `if:` para controlar a execução de um *step* ou *job* com base no resultado das etapas anteriores.

Por padrão, se uma etapa falha, o GitHub Actions **aborta a execução dos passos seguintes**. As funções de status permitem sobrescrever ou personalizar esse comportamento.

Resumo das Funções de Status

Função	Quando a condição é VERDADEIRA?	Uso Principal
<code>success()</code>	Quando nenhuma etapa anterior falhou ou foi cancelada. (<i>Padrão do GitHub Actions</i>)	Etapas padrão da esteira (compilação, testes, deploy).
<code>failure()</code>	Quando pelo menos uma etapa anterior do mesmo job falhou.	Envio de alertas de erro, salvamento de <i>logs</i> de diagnóstico e <i>rollback</i> .
<code>always()</code>	Sempre , independentemente de sucesso, falha ou cancelamento das etapas anteriores.	Etapas de limpeza (<i>cleanup</i>), derrubar contêineres e relatórios de execução.
<code>cancelled()</code>	Apenas quando o <i>workflow</i> foi cancelado manualmente por um usuário ou por <i>timeout</i> .	Notificar a equipe que o processo foi interrompido voluntariamente.

Funções úteis em condicionais

1. SUCCESS (Padrão implícito, mas pode ser explicitado com condições extras)

- name: Deploy Temporário / Notificação de Sucesso

if: success() && github.event_name == 'push'

run: echo "Testes passaram! Preparando para o deploy..."

2. FAILURE: Executa APENAS se algum passo anterior falhar

- name: Upload de Logs e Prints de Erro

if: failure()

uses: actions/upload-artifact@v4

with:

name: relatorio-de-falha

path: ./logs/

retention-days: 7

- name: Envia notificação

if: failure()

run: echo "Alerta enviado: A build falhou na etapa de testes!"

Funções úteis em condicionais

```
# 3. CANCELLED: Executa APENAS se o workflow for cancelado
- name: Tratar Cancelamento
  if: cancelled()
  run: echo "O workflow foi interrompido manualmente pelo operador."

# 4. ALWAYS: Executa SEMPRE (Sucesso, Falha ou Cancelado)
- name: Limpeza de Recursos (Cleanup)
  if: always()
  run: |
    echo "Derrubando contêineres e limpando arquivos temporários..."
    echo "Status final do Job: ${ job.status }"
```